StockSync

Product Batch Traceability and Recall Management System

DBMS Level 3 Project Report

Team Member	SRN
Rekha Dhorigol	PES1UG24CS370
Reet Porwal	PES1UG24CS369

Section: G
Repository: https://github.com/reetporwal66/inventory-traceability

Project overview

StockSync is an inventory traceability system for managing products, suppliers, batches, warehouses, stock levels, reservations, transfers, shipments, quality defects and product recalls. MySQL stores transactional records and enforces relational integrity. MongoDB stores operational event documents synchronized through a MySQL outbox workflow.

1. Abstract

StockSync is a web-based inventory traceability and recall-management system. It supports product and batch management, warehouse inventory, stock reservations, inter-warehouse transfers, shipments, quality-defect reports, recalls, quarantine and recovery. The application is built with Next.js, React and TypeScript. MySQL/InnoDB stores transactional operational data and enforces relational integrity, while MongoDB stores event documents synchronized from a MySQL transactional outbox. The system includes role-aware access for administrators, inventory staff and quality staff. Database views support operational reporting, stored procedures handle multi-step stock operations, and inventory triggers generate outbox events. Application APIs also publish selected product, recipient and shipment events. A continuously running worker processes pending events with retry and reconnection handling.

2. Problem Statement and Objectives

Inventory operations require reliable stock counts, traceability between batches and destinations, and timely handling of shipment discrepancies, defects and recalls. Scattered or inconsistent records can make it difficult to identify affected stock and recipients. StockSync aims to provide a structured data model and operational workflow for inventory control and traceability.

Objectives

- Maintain product, supplier, batch, warehouse and inventory records.
- Enable authorized users to create and maintain product master records with validated fields and unique product identifiers.

• Support batch creation linked to existing products and suppliers, maintaining batch-level traceability.

• Support stock reservation, cancellation and inter-warehouse transfer.

• Track shipments, shipment items, recipients and delivery discrepancies.

• Record quality defects, recalls, affected batches and recovery actions.

• Provide reporting views for inventory status, low stock, recall tracing and shipment discrepancies.

• Publish selected inventory events to MongoDB through an outbox mechanism.

3. Functional Requirements

Area	Requirement
Authentication and access control	Authenticate users and restrict operations according to administrator, inventory staff and quality staff roles.
Product management	View and create products with SKU, category, unit of measure and reorder-level information.
Batch management	Create and view batches linked to products and suppliers, select an initial warehouse and quantity, and initialize corresponding inventory.
Inventory	View stock by batch and warehouse, including on-hand, reserved, quarantined and database-generated available quantities.
Reservations and transfers	Reserve and cancel stock and transfer batches between warehouses while validating inventory quantities.
Shipments and recipients	Create recipients and shipments, manage shipment status transitions, record dispatched/received quantities and identify discrepancies.
Quality and recalls	Record and review defects, initiate recalls, quarantine or release stock, and record recovery actions.
Event synchronization	Record selected events in the MySQL outbox and synchronize them to MongoDB through the event-processing workflow.

4. Technology and Database Design

The application uses Next.js 16.4.2, React 19.3.0 and TypeScript, with Next.js API routes providing server-side application logic. MySQL/InnoDB stores relational operational data, while MongoDB stores event documents in the inventory_events database and events collection. Supporting libraries include mysql2 for MySQL access, mongodb for MongoDB integration, Zod for input validation, bcryptjs for password hashing and jose for session-token handling. A separate Node.js worker continuously processes pending MySQL outbox events and delivers them to MongoDB. Database credentials and session secrets are configured through environment variables.

5. Entity-Relationship Design

The ER design models users, products, suppliers, batches, warehouses, inventory, reservations, transfers, recipients, shipments, shipment items, defect reports, recalls, recall-batch associations, recall recoveries and the event outbox. Products and suppliers are related to batches, while inventory connects batches to warehouses. Shipments reference recipients and source warehouses and contain shipment items linked to batches. Recalls and batches are associated through recall_batches, which uses a composite primary key. recall_recoveries references that association through the composite foreign key (recall_id, batch_id). Inventory enforces uniqueness for each batch–warehouse pair. The event outbox stores event metadata and payloads; its aggregate identifiers are not declared foreign keys.

.

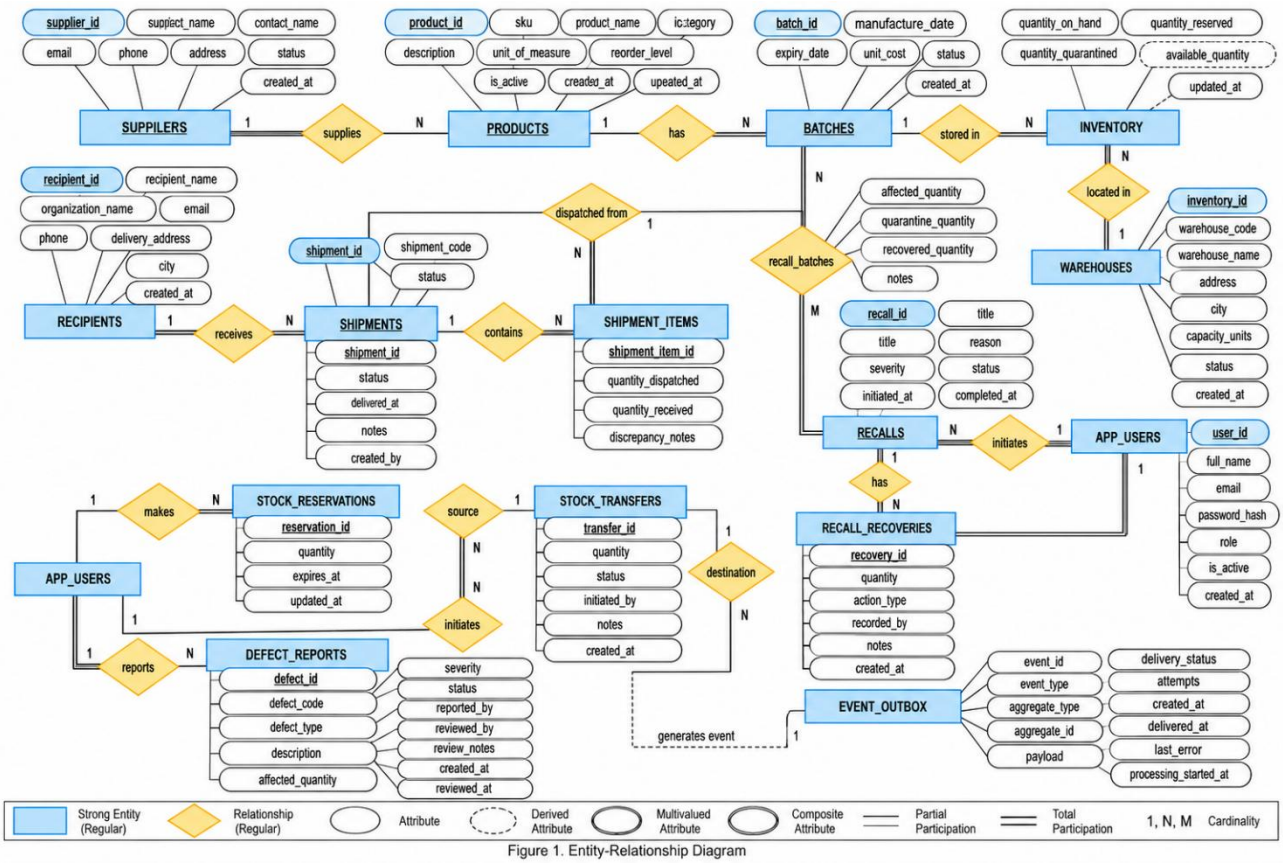


Figure 1. Entity-Relationship Diagram

Figure 1. Entity-Relationship Diagram

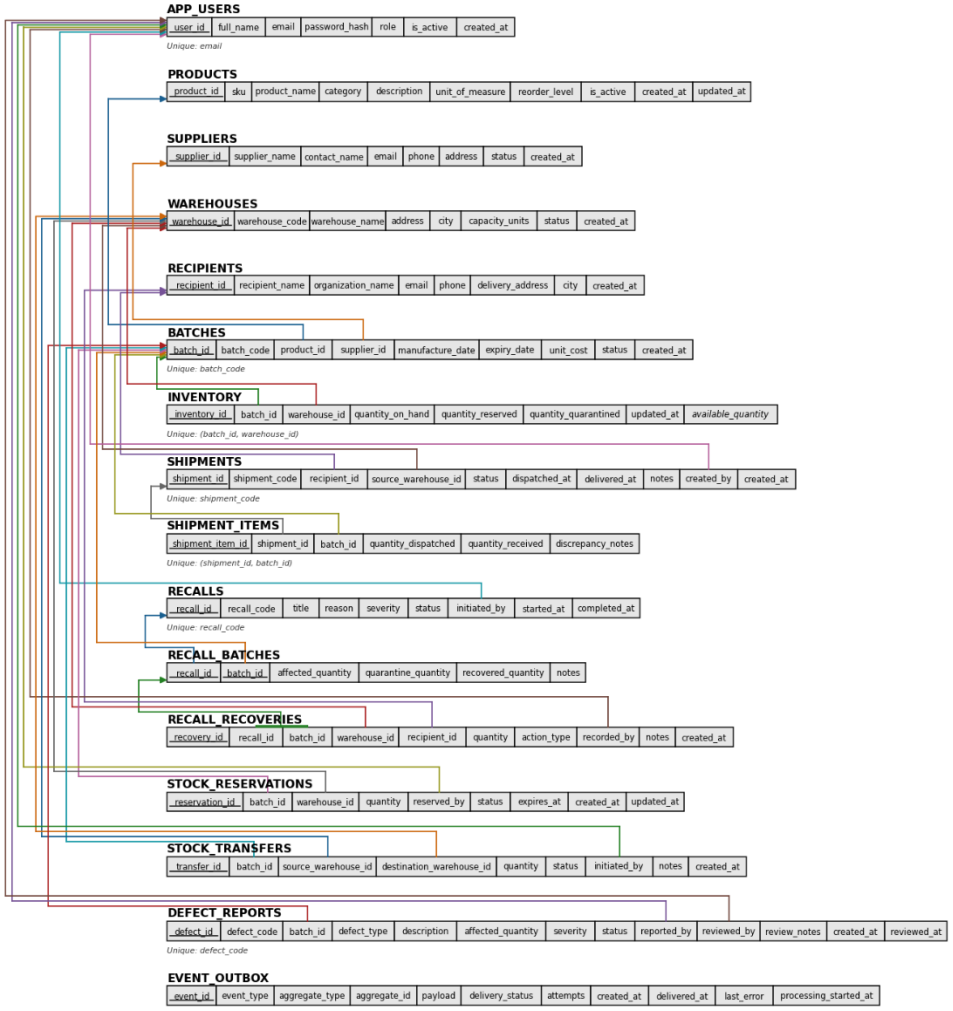
6. Relational Schema

The relational schema represents the system's 16 tables, primary keys, foreign keys, unique constraints and validation rules. Important integrity rules include the composite primary key (recall_id, batch_id) in recall_batches, the composite foreign key from recall_recoveries to recall_batches, and the unique constraints on (batch_id, warehouse_id) in inventory and (shipment_id, batch_id) in shipment_items. The inventory.available_quantity column is generated from on-hand stock minus reserved and quarantined quantities. These constraints help maintain consistency across inventory, shipment, quality and recall operations.

(Diagram on next page)

StockSync — Relational Schema

underline = primary key arrow = foreign key → referenced primary key italic = derived attribute Unique: = alternate (unique) key



StockSync — Relational Schema

PK = primary key (underlined); FK = foreign key; UK = unique key; derived = generated attribute.
Relation attributes follow the schema implemented in MySQL. Composite keys are shown explicitly.

APP_USERS

user_id	PK
full_name	
email	UNIQUE
password_hash	
role	
is_active	
created_at	

PRODUCTS

product_id	PK
sku	UNIQUE
product_name	
category	
description	
unit_of_measure	
reorder_level	
is_active	
created_at	
updated_at	

SUPPLIERS

supplier_id	PK
supplier_name	
contact_name	
email	
phone	
address	
status	
created_at	

BATCHES

batch_id	PK
batch_code	UNIQUE
product_id	FK→PRODUCTS
supplier_id	FK→SUPPLIERS
manufacture_date	
expiry_date	
unit_cost	
status	
created_at	

WAREHOUSES

warehouse_id	PK
warehouse_code	UNIQUE
warehouse_name	
address	
city	
capacity_units	
status	
created_at	

INVENTORY

inventory_id	PK
batch_id	FK→BATCHES
warehouse_id	FK→WAREHOUSES
quantity_on_hand	
quantity_reserved	
quantity_quarantined	
available_quantity	DERIVED
updated_at	
(batch_id, warehouse_id)	UNIQUE

STOCK_RESERVATIONS

reservation_id	PK
batch_id	FK→BATCHES
warehouse_id	FK→WAREHOUSES
quantity	
reserved_by	FK→APP_USERS
status	
expires_at	
created_at	
updated_at	

STOCK_TRANSFERS

transfer_id	PK
batch_id	FK→BATCHES
source_warehouse_id	FK→WAREHOUSES
destination_warehouse_id	FK→WAREHOUSES
quantity	
status	
initiated_by	FK→APP_USERS
notes	
created_at	

RECIPIENTS

recipient_id	PK
recipient_name	
organization_name	
email	
phone	
delivery_address	
city	
created_at	

SHIPMENTS

shipment_id	PK
shipment_code	UNIQUE
recipient_id	FK→RECIPIENTS
source_warehouse_id	FK→WAREHOUSES
status	
dispatched_at	
delivered_at	
notes	
created_by	FK→APP_USERS
created_at	

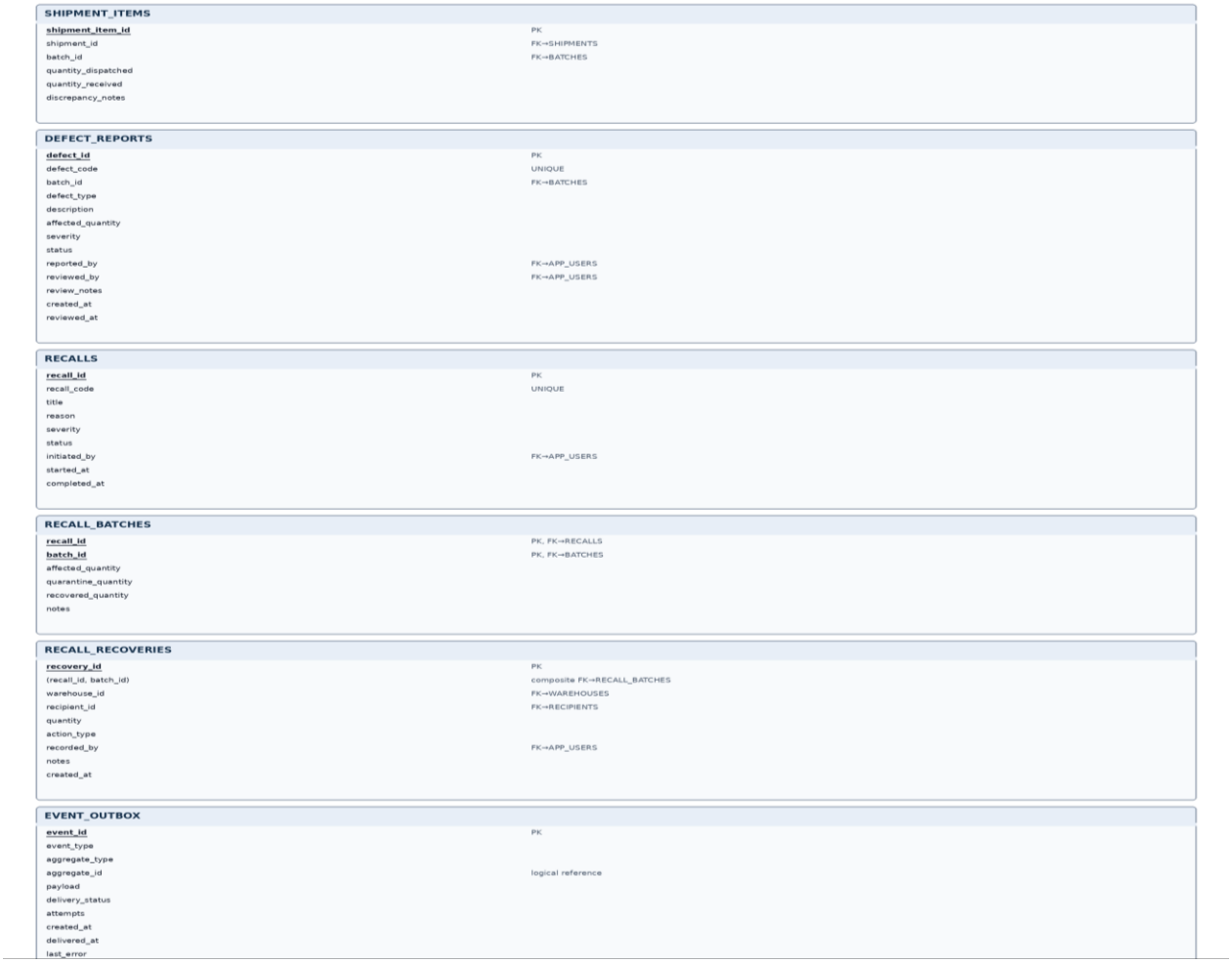


Figure 2. Relational Schema

7. Data Dictionary Summary

Table	Purpose / key attributes
app_users	Application users and role-related information.
products	Product master records, unique SKU, product name, category, unit, reorder level and product status.
suppliers	Supplier information associated with sourced products/batches.
batches	Batch-level traceability; product_id and supplier_id foreign keys; batch_code UNIQUE; manufacture/expiry dates; unit_cost and status. Batch creation initializes inventory for the selected warehouse and quantity
warehouses	Warehouse/location master records.
inventory	inventory_id PK; batch_id and warehouse_id FKs; on-hand/reserved/quarantined quantities; generated available_quantity; UNIQUE(batch_id, warehouse_id).
stock_reservations	Reservations of inventory and their lifecycle status.
stock_transfers	Records of stock movement between locations.
recipients	Shipment recipient details.
shipments	Shipment status and shipment-level information.
shipment_items	shipment_item_id PK; shipment_id and batch_id FKs; dispatched/received quantities; discrepancy notes; UNIQUE(shipment_id, batch_id).
defect_reports	Defect codes, details, severity/status and related batch.
recalls	Recall records and their status/details.
recall_batches	Association between recalls and affected batches.
recall_recoveries	Recorded recovery actions and outcomes.
event_outbox	Events queued for downstream delivery and delivery status.

8. Integrity, Indexing and Advanced SQL

Primary keys identify records, while foreign keys connect related entities and help prevent orphaned records. Unique constraints protect business identifiers where defined. Transactional procedures centralize operations that must update related stock records consistently.

Stored procedures

• **sp_reserve_stock** — reserves stock according to the procedure's validation and update logic.

• **sp_cancel_reservation** — cancels a reservation and updates related reservation quantities/status.

• **sp_record_quarantine** — records stock quarantine as part of quality and recall handling.

• **sp_transfer_stock** — handles stock transfer between locations.

Reporting views

• **vw_inventory_overview** — summarizes inventory information.

• **vw_low_stock** — identifies items requiring low-stock attention.

• **vw_recall_recipient_trace** — supports tracing recall-related shipments and recipients.

• **vw_shipment_discrepancies** — exposes shipment discrepancy information.

Integrity & Constraints

Primary keys identify records, while foreign keys maintain referential integrity between related tables. Unique constraints protect identifiers such as user email, product SKU, batch code, warehouse code, shipment code, recall code and defect code. Composite unique constraints prevent duplicate inventory records for a batch–warehouse pair and duplicate batch entries within a shipment. CHECK constraints validate applicable values and business rules, including non-negative unit cost, batch date order, inventory quantity bounds, reservation and transfer quantities, distinct transfer warehouses and shipment date order.

The generated column inventory.available_quantity is calculated as quantity_on_hand - quantity_reserved - quantity_quarantined.

Triggers and transactional outbox

The database defines trg_inventory_insert_outbox and trg_inventory_update_outbox to create outbox events for applicable inventory insert and update operations. In addition, application APIs create outbox events for selected operations, including product creation, recipient creation and shipment creation. The related database changes and outbox inserts are grouped transactionally where implemented, reducing the risk of committing an operation without recording its corresponding event.

The event_outbox table tracks event type, aggregate metadata, JSON payload, delivery status, attempt count and processing timestamps. Its aggregate_type and aggregate_id fields identify the related entity but are not declared foreign keys.

Background Processing

The separate Node.js worker continuously polls pending or retryable outbox events, writes corresponding documents to MongoDB using event_id as an idempotency key, and updates delivery status. The worker includes retry limits, recovery of stale processing records, connection reconnection and graceful shutdown handling. Successful delivery still depends on the configured databases being available and the worker running correctly.

9. MongoDB Event Model

StockSync uses MongoDB for event-oriented documents in the inventory_events database and events collection, while MySQL remains the source of truth for transactional inventory and traceability data. Inventory triggers and selected application API operations create event records in the MySQL event_outbox table. The separate Node.js worker continuously checks for pending or retryable events, claims events for processing, writes documents to MongoDB and updates their delivery status in MySQL.

The worker uses event_id as an idempotency key to avoid creating duplicate event documents when an event is retried. It also implements retry limits, recovery of stale PROCESSING records, reconnection handling and graceful shutdown. The application additionally provides authenticated event viewing and an administrator-restricted event synchronization API. These mechanisms support asynchronous event delivery and inspection; successful end-to-end delivery must be confirmed using actual worker logs, MySQL outbox status and MongoDB documents.

10. Testing and Evidence

The project includes evidence for the main database and application workflows. The following checks describe the expected behavior covered by the project test scenarios.

Test area	Expected result
Build and lint	Application linting and production build complete without errors.
Reservation lifecycle	Creating and cancelling a reservation restores the reserved quantity to the expected value.
Shipment status	Valid status transitions succeed; invalid transitions are rejected with an appropriate conflict response.
Recall and quarantine	Quarantine and recall actions create the corresponding traceability and recovery records.
MySQL-to-MongoDB synchronization	Pending outbox events are written to MongoDB and their delivery status is updated.
Product creation	A valid product is saved and appears in the product listing.
Duplicate SKU validation	A duplicate SKU is rejected with a conflict response.
Batch creation	A valid batch is saved with its product and supplier associations.
Batch validation	Invalid batch details or invalid product/supplier associations are rejected.
Product event recording	Product creation and its outbox event are committed together; a failed transaction does not leave a partial product creation.
Static checks	ESLint and the production build complete successfully.
Product and batch creation	A valid product can be created, and creating a batch with a selected warehouse and positive initial quantity creates the batch and corresponding initial inventory consistently.
Outbox worker recovery	Pending events are delivered to MongoDB; duplicate processing does not create duplicate documents for the same event ID, and stale processing records can be recovered for retry.

11. Setup and Operational Considerations

Use the project README and .env.example to configure the application and its database connections. Set environment variables for the local environment rather than committing secrets. Import the SQL setup into a dedicated development database and confirm the database user has the privileges required by the setup and application.

The SQL setup script contains destructive table-reset statements and sample-data operations. It should be run only against an intended development/test database after reviewing the target connection. The script also

changes foreign-key-check behavior during import; verify the resulting schema and data integrity after setup. Definer clauses may require adjustment for a local MySQL account.

The application and background worker require their respective environment variables and database connectivity. The worker must be started separately using the repository's npm run worker command for continuous outbox processing.

Project evidence

Screenshots documenting the user interface, MySQL queries and MongoDB event records are maintained separately in the project's **docs/screenshots/** directory. This report includes only the ER diagram and relational schema as figures.

12. Conclusion

StockSync combines relational inventory management and product-batch traceability with reservation, transfer, shipment, quality and recall workflows. Its MySQL schema uses keys, constraints, generated values, reporting views, stored procedures and inventory outbox triggers to support data integrity and multi-step operations. The application provides role-aware access and product/batch management, including initialization of inventory during batch creation. Selected inventory and application events are recorded in a transactional outbox and processed by a separate worker for storage in MongoDB. Together, these components provide an integrated foundation for operational traceability and recall management, with end-to-end behavior verified through application, SQL and MongoDB evidence.